

Arynik1 App Summary (Repo-Based, One Page)

What It Is

Postis Shipment Status PWA is a mobile-first delivery operations app for scanning AWB labels and updating shipment statuses. The repo implements a React/Vite installable PWA frontend and a FastAPI backend that authenticates users and integrates with Postis APIs.

Who It Is For

Primary persona: courier/driver who scans parcels and submits field status updates. Secondary personas in RBAC: Admin, Manager, Dispatcher, Warehouse, Support, Finance, Viewer, Recipient.

What It Does

- AWB scanning with camera barcode/QR detection plus manual entry fallback.
- Status update workflows that call secured backend endpoints (/update-awb, /shipments/update-status).
- Offline-first queue in IndexedDB with automatic re-sync when connectivity returns.
- Role-based access control with JWT sessions and permission-gated screens/endpoints.
- Operations modules for manifests, route plans/runs, live driver location, tracking requests, and chat.
- Snapshot fallback mode when backend is unreachable, with API auto-detect from Settings.

How It Works (Architecture)

- Client layer: React + Vite + HashRouter PWA (frontend/src/main.jsx, frontend/src/App.jsx, frontend/vite.config.js).
- Client services: centralized API client and data-source switching (frontend/src/services/api.js) plus offline queue (frontend/src/store/queue.js).
- API layer: FastAPI app with OAuth2/JWT auth, CORS, and permission checks (backend/main.py, backend/authz.py).
- Data layer: SQLAlchemy models/session (backend/models.py, backend/database.py); defaults to SQLite locally and supports Postgres via DATABASE_URL.
- Integration layer: Postis client for auth/tracking/status updates and background Postis sync loop (backend/postis_client.py, backend/services/postis_sync_service.py).
- Data flow: scan AWB -> frontend sends or queues update -> backend validates role and logs -> backend updates DB and calls Postis -> UI reflects sync/result state.

How To Run (Minimal)

- Install backend deps from repo root: `pip install -r backend/requirements.txt`.
- Create backend/.env from .env.example and set at least POSTIS_USERNAME, POSTIS_PASSWORD, DATABASE_URL, and JWT_SECRET.
- Seed initial admin and run API: `python seed_db.py` then `uvicorn backend.main:app --reload`.
- Start frontend: `cd frontend && npm install`; set VITE_API_URL in .env; then run `npm run dev`.

Missing Info

Not found in repo: a dedicated architecture diagram; architecture above is code-derived.

Repo Evidence

Evidence files: README.md, backend/main.py, backend/authz.py, backend/models.py, backend/database.py, backend/postis_client.py, backend/services/postis_sync_service.py, frontend/src/main.jsx, frontend/src/App.jsx, frontend/src/services/api.js, frontend/src/store/queue.js, frontend/vite.config.js, render.yaml